

Supplementary Materials

Materials and methods

Commands

PASTA 1.8.3

```
python3 run_pasta.py -i sequences -o output_dir
-t start_tree --iter-limit iterations
--temporaries temp_dir
```

Comment: PASTA always uses its default starting tree, but we run it for varying numbers of iterations.

MAGUS (Fast)

```
python3 magus.py -d working_dir -o result
-i sequences --graphbuildhmmextend false
--maxsubsets 0 --maxnumsubsets num_subsets
-r num_backbones -m backbone_size
```

MAGUS (Slow)

```
python3 magus.py -d working_dir -o result
-i sequences --graphbuildhmmextend true
--maxsubsets 0 --maxnumsubsets num_subsets
-r num_backbones -m backbone_size
```

Comment: MAGUS is available in open source form in Github. The decomposition repeats until one or both of the following constraints hold: (1) the number of subsets is at least maxnumsubsets, and (2) the maximum subset size is at most maxsubsets; hence, this command makes the decomposition produce 25 subsets, without concern for size.

HMMER 3.1b2 (called within MAGUS)

```
hmmbuild --ere 0.59 --cpu 1 --symfrac 0.0
--informat afa hmm_model alignment
```

```
hmmalign -o result hmm_model sequences
```

Comment: this is used for the HMM-extension calculation, described below.

MCL 14-137 (called within MAGUS)

```
mcl alignment_graph --abc -o clusters -I 4
```

Computing Alignment Error - FastSP 1.6.0

```
java -jar FastSP_1.6.0.jar -r
true_alignment -e estimated_alignment
```

Additional Details

Runtime Complexity Because GCM uses a local search heuristic in its design, its running time does not have a nice Big-O form, but a discussion of the different steps is informative about its worst case and typical case running time. GCM's steps have the following components: 1) generating backbone alignments, 2) compiling the alignment graph, 3) clustering the graph, and 4) fixing/ordering the clustering into a trace.

The first step involves generating the backbone alignments, where we used 10 runs of MAFFT -linsi. The running time of MAFFT -linsi is $O(N^2 L^2)$, where N is the number of sequences and L is their length (Kato and Toh, 2008). Then, if B_n is the number of backbone alignments, B_l is their average length, and B_m is the maximum number of sequences in each backbone, we have $O(B_n B_m^2 B_l^2)$.

After we compute the backbone alignments, constructing the alignment graph is $O(B_n B_l B_m^2)$: this follows from reading all B_m^2 residue pairs in each backbone alignment column, of which we have $B_n B_l$.

Clustering the graph is done by invoking the Markov Clustering software. According to Azad *et al.* (2018), the runtime of MCL is optimized to $O(V d^2)$, where V is the number of nodes, and d is their average degree. Let S denote the number of constraint alignments being merged and C be the average number of columns per constraint alignment. In the best case, if our alignment graph is sparse, we would have SC nodes with an average degree of $O(S)$, giving us a runtime of $O(S^3 C)$. In the worst case, however, the runtime will be $O(SC(B_n B_m)^2)$, since the worst case would match every constraint alignment column with B_m different neighbors in each of B_n backbone alignments. Informally spot-checking some of our graphs in this study has shown sparsities of over 99%, so such degenerate cases are unlikely to appear empirically.

The last phase (fixing/ordering the clusters to build the trace) is somewhat more difficult to characterize, as it relies on a heuristic that does not have a natural Big-O asymptotic complexity. However, it seems reasonable to say that the A* algorithm's naive time complexity is exponential in the worst case: $O(S^{SC})$, since S is the worst-case branching factor in our search space, the SC is the worst-case length of a path from the start state to an end state. To prevent this worst case, we impose a hard constant limit on the number of states that can be explored before the algorithm is forced to make progress—forced to order a cluster irreversibly. Thus, the worst-case runtime of this step within GCM is $O(SCF)$, where SC is the length of the longest possible search path and F is a constant that roughly describes the maximum number of non-progressing states that can be explored before the algorithm is forced to advance along this path.

Comments on MCL To use MCL in GCM, we normalize the adjacency matrix for the graph G so that it defines a stochastic matrix (i.e., the entries in the matrix are non-negative and the column sums are 1). MCL alternates between “expansion” and “inflation” phases. The “expansion” phase simply multiplies this matrix by itself, which propagates the random walk probabilities between nodes. In fact, this operation is very similar to the consistency propagation operation in Probcons, a method for amino acid sequence alignment (Do *et al.*, 2005), where the matrix of pairwise probabilities between residues was repeatedly multiplied to propagate consistency. MCL allows us to tap into the same mechanism to take advantage of the same principle. The “inflation” step serves to gradually put more emphasis on more probable paths and less emphasis on less probable paths, driving the algorithm to converge to a clustering.

The chief algorithmic parameter in MCL is the inflation factor I , which governs the “inflation” operator and roughly controls the granularity of the clustering. The guidelines about choosing I in Von Dongen (2012) says “A good set of starting values is 1.4, 2, 4, and 6”; our preliminary exploration (Figure 1) using an early version of our approach on one replicate of one of our model conditions showed that $I = 1.4$ produced poor results, and $i = 2, 4, 6$ were very similar; we set $I = 4$ for all subsequent studies.

HMM-Extension (Slow vs. Fast Mode) GCM “slow” mode extends the backbone alignments to contain all of the input sequences, and it does this using HMMER (Eddy, 2020). The process is nearly identical to the initial alignment procedure: after our 200 sequences are aligned with MAFFT, we use hmmbuild to produce an HMM over this alignment, and hmmalign to add all of the remaining sequences to it.

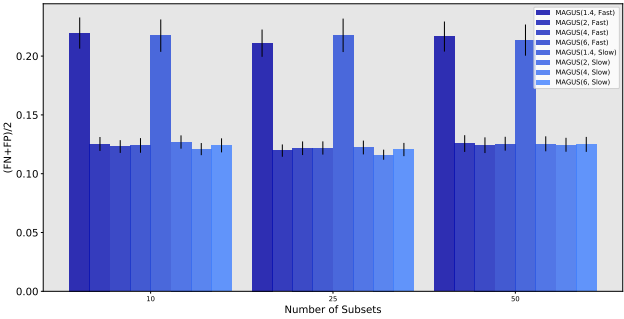


Fig. 1. Experiment 1: Exploring impact of the MCL inflation parameter. MAGUS(X, Y) indicates MAGUS being run with inflation parameter X and mode Y. The error rates are the average of SPFN and SPFP, and are averaged across the 19 replicates of the 1000M1 model condition. Error bars indicate standard error. The x-axis shows the impact of changing the number of subsets.

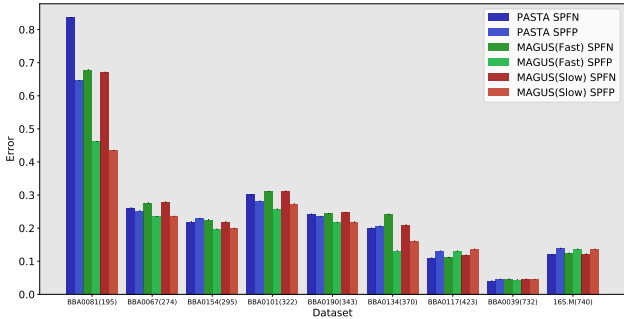


Fig. 4. Experiment 2: Error rates for MAGUS and PASTA on 8 small datasets from Balibase and 16S.M. The error rates are SPFN and SPFP. The x-axis shows the dataset, with the number of sequences in parentheses. These datasets were modified to remove sequences with length more than 20% above or below the median. PASTA and MAGUS were both run in default mode.

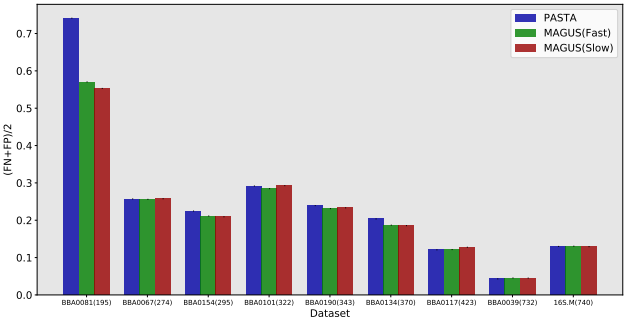


Fig. 2. Experiment 2: Error rates for MAGUS and PASTA on 8 small biological datasets from Balibase and 16S.M. The error rates are the average of SPFN and SPFP. The x-axis shows the dataset, with the number of sequences in parentheses. These datasets were modified to remove sequences with length more than 20% above or below the median. PASTA and MAGUS were both run in default mode.

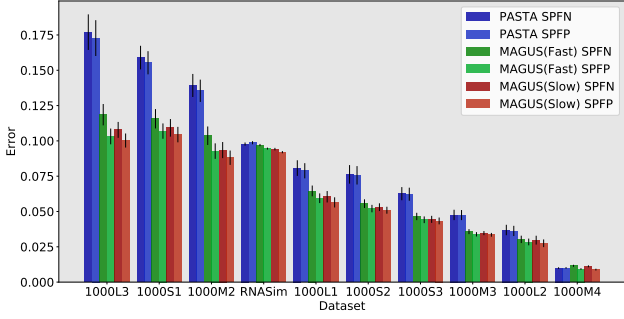


Fig. 5. Experiment 2: Error rates for MAGUS and PASTA on the simulated 1000-taxon nucleotide datasets. The error rates are SPFN and SPFP, averaged over 20 replicates. Error bars indicate standard error. The x-axis shows the dataset. PASTA and MAGUS were both run in default mode.

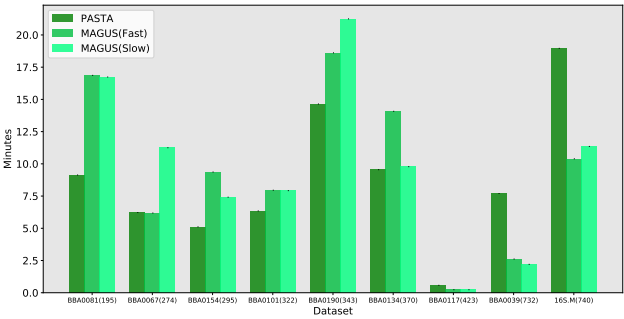


Fig. 3. Experiment 2: Running time (minutes) for MAGUS and PASTA on 8 small datasets from Balibase and 16S.M. The x-axis shows the dataset, with the number of sequences in parentheses. These datasets were modified to remove sequences with length more than 20% above or below the median. PASTA and MAGUS were both run in default mode.

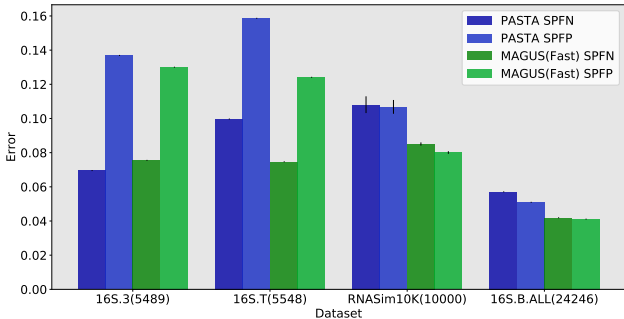


Fig. 6. Experiment 2: Error rates for MAGUS and PASTA on on the larger nucleotide datasets. The error rates are SPFN and SPFP. RNASim is simulated (averaged over 10 replicates, error bars indicate standard error); the other datasets are biological and were modified to remove sequences with length more than 20% above or below the median. The x-axis shows the dataset, with the number of sequences in parentheses. PASTA and MAGUS were both run in default mode.

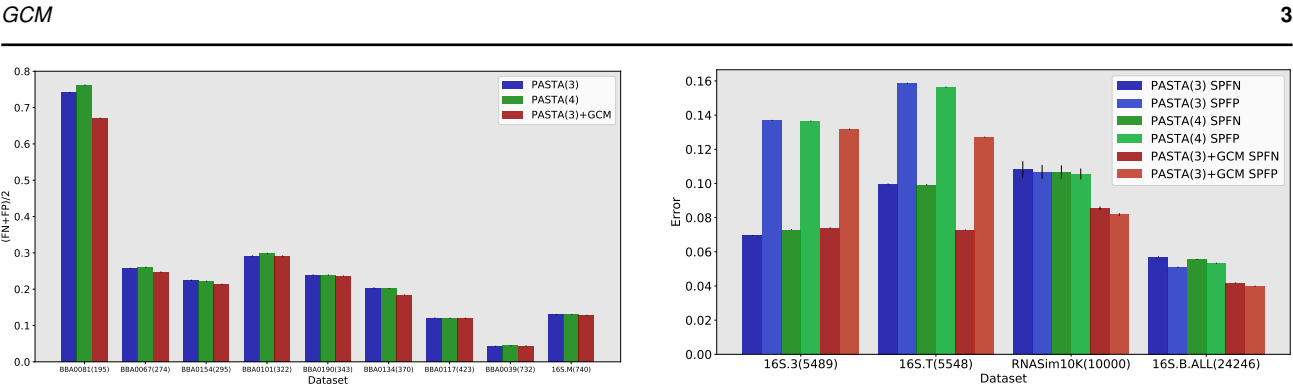


Fig. 7. Experiment 3: Error rates for PASTA(3),PASTA(4), and PASTA(3)+GCM on small biological datasets from Balibase and 16S.M. The error rates are the average of SPFN and SPFP. The x-axis shows the dataset, with the number of sequences in parentheses. These datasets were modified to remove sequences with length more than 20% above or below the median. GCM was run in slow mode.

Fig. 10. Experiment 3: Error rates for PASTA(3), PASTA(4), and PASTA(3)+GCM on the larger nucleotide datasets. The error rates are SPFN and SPFP. RNASim is simulated (averaged over 10 replicates, error bars indicate standard error); the other datasets are biological and were modified to remove sequences with length more than 20% above or below the median. The x-axis shows the dataset, with the number of sequences in parentheses. PASTA was run in default mode. GCM was run in fast mode.

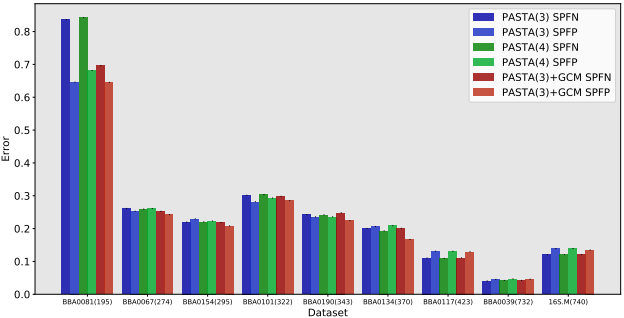


Fig. 8. Experiment 3: Error rates for PASTA(3), PASTA(4), and PASTA(3)+GCM on 8 small datasets from Balibase and 16S.M. The error rates are SPFN and SPFP. The x-axis shows the dataset, with the number of sequences in parentheses. These datasets were modified to remove sequences with length more than 20% above or below the median. PASTA was run in default mode. GCM was run in slow mode.

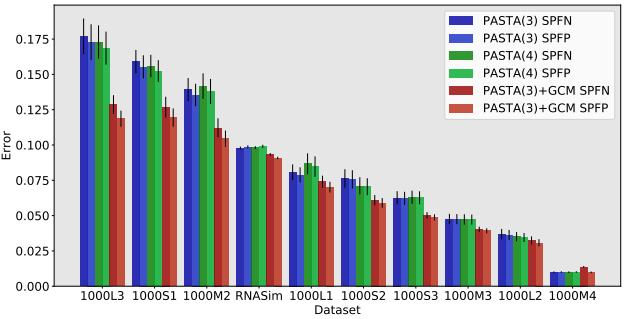


Fig. 9. Experiment 3: Error rates for PASTA(3), PASTA(4), and PASTA(3)+GCM on the simulated 1000-taxon nucleotide datasets. The error rates are SPFN and SPFP, averaged over 20 replicates. Error bars indicate standard error. The x-axis shows the dataset. PASTA was run in default mode. GCM was run in slow mode.

References

Azad, A., Pavlopoulos, G. A., Ouzounis, C. A., Kyrpides, N. C., and Buluç, A. (2018). HipMCL: a high-performance parallel implementation of the Markov clustering algorithm for large-scale networks. *Nucleic Acids Res*, **46**(6), e33–e33.

Do, C. B., Mahabhashyam, M. S., Brudno, M., and Batzoglou, S. (2005). Probcons: Probabilistic consistency-based multiple sequence alignment. *Genome Res*, **15**(2), 330–340.

Eddy, S. R. (2020). HMMER website. <http://hmmer.org/>, last accessed September 25, 2020.

Kato, K. and Toh, H. (2008). Recent developments in the MAFFT multiple sequence alignment program. *Brief Bioinf*, **9**(4), 286–298.

Von Dongen, S. M. (2012). MCL manual. <https://micans.org/mcl/man/mcl.html>.